

Lezione 19/05

Copy-On-Write

Tecnica di cui anche i file-system fanno uso, lo vedremo in seguito. Questa è una tecnica di ottimizzazione, che permette di risparmiare memoria in situazioni in cui processi diversi hanno delle pagine che hanno, in una data fase, lo stesso contenuto.

Dopo una fork, i due processi hanno sostanzialmente gli stessi contenuti. Poi il contenuto del processo figlio verrà sostituito tramite il comando exec. In questo modo faccio puntare alla tabella delle pagine del processo figlio, i frame del padre. Così devo solo creare questa nuova entità (processo), e non avrò bisogno di duplicare il contenuto dello spazio di indirizzamento, ma semplicemente farò puntare le voci della tabella delle pagine del nuovo processo ai frame del processo padre. Questo naturalmente riguarda solo alcune pagine, quelle condivise, non tutte.

Diverso dalle pagine condivise già viste: cioè, i processi non hanno chiesto di condividere la pagina, ognuno ne ha chiesta una copia indipendente. È il SO che fa un'astrazione facendo leggere a entrambi i processi la stessa identica pagina. Naturalmente, questo funziona finché non viene fatta alcuna modifica. Bisogna gestire il caso in cui P1 e P2 richiedano di fare modifiche del frame. Che succede? Il SO "duplica" il contenuto, fornendo una copia personale di tale frame a entrambi i processi, così il processo richiedente potrà effettuare la write. Naturalmente questa duplicazione avviene solo per la pagina su cui è stata richiesta la scrittura, non su tutto lo spazio di indirizzamento. Per questo il nome **copy-on-write**: non appena viene richiesta una scrittura, fanno una copia. Quindi fai una copia solo quando sta per essere effettuata una scrittura.

Il sistema operativo, per intercettare le operazioni di scritture, aveva creato una maschera per cui quella pagina era accessibile in sola lettura. Il SO, quando arriva una richiesta di scrittura, dovrà rimediare: creare la copia e ripristinare i permessi originali. Ciò che non viene modificato, resta condiviso. Quantità molto più ampia di quanto si possa pensare.

Come funziona? L'istruzione che ha causato problemi, in quanto ha trovato una pagina in sola lettura, viene rilanciata (e così poi andrà a buon fine), come accade

per i page fault. Naturalmente, i permessi originali, dato che non si trovano nella tabella delle pagine dei processi, si trova in strutture dati di supporto. Nel momento in cui non c'è più la condivisione, viene ripristinata la maschera originale.

Zero-Fill-On-Demand

Heap del processo gestito dall'allocatore. Allocatore viene inserito nelle librerie standard da parte del compilatore. Hanno il compito di sapere quali blocchi sono liberi, quali sono allocati, etc. Si ripropone un problema già visto. Qui, il fatto che ci siano dei buchi nella memoria non ci pesa più di tanto. Problema meno preponderante in termini di spreco, perché nel breve termine, dato che anche qui funziona tramite paginazione, alcune pagine verranno liberamente dato che si svuoteranno. La cosa più rapida qui, invece di cercare buchi/buchini, è alzare l'asticella dell'heap e chiedere al SO di chiedere qualche byte/kbyte di memoria in più, da prendere dallo spazio vuoto. Quando riutilizzo parti di memoria usate già nell'heap, devo stare attento, perché quei blocchi sono già stati usati e quindi probabilmente "sporcati" da altri dati. Quindi per questo con malloc dobbiamo stare attenti a impostarli a 0. Il SO, infatti, quando viene richiesta nuova memoria (alzando l'asticella), deve svuotare le pagine nuove, deve pulirle. E quindi lasciare le pagine nuove dovranno essere vuote, pulite, tutto impostato a 0. Il SO deve svuotarle prima di assegnarle al processo, anche per evitare problemi di privacy/sicurezza, dato che quelle pagine potevano appartenere a un altro processo. E se non ho RAM libera? Dovrò chiamare l'algoritmo di sostituzione delle pagine. Es. se chiedo 100 pagine, dovrò liberare 100 pagine.

Ottimizzazione: potremmo applicare una sorta di condivisione su una pagina speciale, nota come **read-only static zero page**. Questo è un frame presente nel nostro SO assolutamente vuoto, pieno di zeri. Quando il processo chiede le sue 100 pagine vuote, allora il processo predispone tabella delle pagine e maschere per fargliele utilizzare. In realtà, questo è fittizio: implemento le 100 pagine del processo dentro la pagina read-only. Tutte le 100 puntano alla stessa pagina. Questo discorso vale per qualunque processo che abbia delle pagine vuote. Qualsiasi processo guarda a questa pagina vuota, per rispondere all'**immediata richiesta di pagine vuote**. Infatti, tutti i riferimenti a questa pagina sono in sola lettura. Questo perché non è detto che la corsa alle 100 pagine sia necessaria subito. Nel momento in cui il processo usa le pagine, però, viene fatta una vera richiesta per avere delle pagine vuote. Comportamento simile al copy-on-write.

Questo lo faccio quando è strettamente necessario, quando il processo sta per usare quella pagina. Potrei fare un'ottimizzazione. Avere un pool di pagine libere, completamente vuote, così da assegnarle non appena vengono richieste. Questo procedimento deve farlo il SO, non l'allocatore, sia per questioni di privacy/sicurezza, e sia perché il SO ha strumenti migliori, come strumenti hw etc. Es. controllo del DMA, che legge e scrive da e per la memoria centrale non usando la CPU. Vale per casi speciali, ma possiamo immaginare che venga utilizzato anche per impostare a 0 tutte le word della pagina. Quando il SO si ritrova con la CPU libera, senza troppo da fare, può eseguire questo task secondario, va a creare/popolare il pool di pagine vuote (sicuro? vale anche per questo?) e ad assegnare la pagina vuota al processo che a breve la richiederà (dato che magari punta alla read-only page). Questa tecnica di zero-fill e anche quella di copy-on-write è usata da tutti i sistemi operativi.

Librerie condivise e file mappati

Librerie condivise da più processi, utili per evitare di reinventare la ruota. Molte di quelle che useremo sono funzioni di libreria, che implementano una serie di funzionalità, e non fanno parte del SO.

Vantaggi: sia per evitare di dover reinventare la ruota e quindi magari fare errori, favorisce riuso del codice in modo virtuoso. Vado a creare un FAT binary exec, pieno di funzioni prese dalla libreria binaria stessa, che però ha determinati vantaggi e svantaggi. Il programma da un lato diventa più grande. Il linker predispone un binario che non include il codice della libreria, ma permette al codice di collegarsi alla libreria, letteralmente creare un binario, per poter invocare le funzioni della libreria, ma lasciando nel binario solo le parti di codice speciali del programma, e lasciare che il codice della libreria condivisa a cui si fa riferimento, [...].

Idea di base: se c'è un processo che ha bisogno di una libreria, questa viene portata in RAM. Due processi totalmente diversi possono avere librerie comuni (es. un browser e una calcolatrice che condividono la libreria per il menù). Il vantaggio è che il processo diventa più piccolo, e lo sviluppo è più semplice. Si ottengono oggetti indipendenti, gruppi indipendenti, sviluppati da programmatori indipendenti.

I nuovi programmi possono usare la nuova libreria senza bisogno di ricompilare i propri codici se uso il linking dinamico. Se viene creata una nuova versione della libreria e uso linking statico, sono costretto a ricompilare tutto. Si può fare nel momento in cui l'aggiornamento non introduce incompatibilità.

Ci sono casi in cui le librerie sono costrette a rompere la retro-compatibilità, e in questo caso è necessario ricompilare i rispettivi processi. A volte capita per grandi upgrade etc. Tipicamente non è obbligatorio aggiornare, i SO ovviamente possono gestire varie versioni.

L'idea della libreria condivisa si può configurare anche in uno scenario per il risparmio di memoria. Avendo un'unica copia della libreria (e senza fare linking statico), i vari processi fanno riferimento a un'unica copia della libreria. Come fa il binario, che parte dal compilatore, a puntare al posto giusto e al momento giusto? Funziona tramite stub. Questo stub inizialmente avrà il compito di richiedere al SO di caricare una copia della libreria, se già non è in memoria; se quella libreria è già stata caricata in memoria, a questo punto lo stub si occuperà di caricare se stesso. Una volta che si usa un stub, questo si occupa di puntare al pezzo giusto della libreria.

Meccanismo che permette a un processo di mappare file in memoria: strumento moderno e utile per accedere al contenuto di un file. Modalità di accesso peculiare che sfrutta la modalità di gestione dei moderni SO chiedendo al SO di mappare i contenuti del file all'interno del processo stesso.

Supponiamo di avere il nostro spazio di indirizzamento e di avere all'interno del nostro disco un file *f*, formato da 4 blocchi. Il processo potrebbe chiedere al SO di mappare il file da qualche parte all'interno del processo. Il SO si occupa di mappare, proiettare, questi 4 blocchi, ovvero 4 pagine. Quando parlo di mappatura/proiezione, sto individuando le 4 pagine. Le relative voci nella tabella delle pagine verranno segnate per indicare che quelle pagine sono valide ma non sono in RAM, dato che sono su disco. Quindi bit di presenza/assenza impostato a 0. Quando il processo *P* farà riferimento a una di queste pagine allora scatterà il page fault. Accade quello che accade solitamente col page fault, solo che stavolta il contenuto è una parte del file, e lo sto mettendo esattamente dove se lo aspetta il processo. Sfrutto il meccanismo di gestione dei page fault per accedere a un file. Questo farò anche per le altre pagine del file. File contiguo, ma in termini di pagine virtuali può non esserlo. Usiamo chiamata di sistema **mmap**, e all'interno

di questo range di indirizzi il processo sa che lì, per lui c'è il file. Andare all'interno di quello spazio di memoria significa quindi accedere al file. Questa cosa funziona benissimo anche con file enormi, di cui magari leggo solo alcune parti. Es. file di 1 GB, e ho una mappatura di 1 GB, ma poi uso solo poche pagine. Posso in questo modo portare un intero database in RAM, mappandolo. In realtà però sto portando in RAM solo alcune parti che mi interessano. A un certo punto, se il processo distoglie la sua attenzione da quel contenuto, allora quel contenuto verrà scartato dalla RAM per le stesse logiche e gli stessi meccanismi in questione.

Quando modifico alcune parti di quel file, sto sporcando il file, e quindi dirty bit a 1. Così, nel momento in cui il processo termina e distoglie la sua attenzione da quel contenuto, quando quella pagina sarà scartata dalla RAM, quella pagina verrà notata che è sporca, e allora verrà pulita e riprotata su disco. In questo modo stiamo modificando un file su disco semplicemente grazie alla paginazione.

Tutti i SO offrono questo tipo di strumenti, anche se con peculiarità a volte diverse.

Tutte le parti della libreria di cui il processo P1 non ha bisogno, potenzialmente non verranno neppure lette dal disco, portando in RAM solo le parti che gli interessano. Se da P2 vengono richieste funzioni diverse, scatta una lettura ad hoc, portando in RAM solo quello che gli interessa.

Altro caso d'uso: possiamo mappare il file binario sulla zona di code. Predispone la tabella delle pagine in modo tale da mappare l'eseguibile in questa zona bassa dello spazio di indirizzamento. Spazio di indirizzamento predisposta in modo tale che la parte bassa corrisponda al binario, portando le parti dell'eseguibile di cui effettivamente si ha bisogno. [Da rivedere].

Nella parte alta, fra heap e codice, avevamo detto c'erano dati inizializzati e con una inizializzazione di default. Queste informazioni devono effettivamente essere scritte all'interno dell'eseguibile insieme al codice eseguibile del programma stesso. Nella parte bassa c'è l'eseguibile, nella parte alta ci sono dati che hanno un valore. Nella parte bassa, quella che corrisponde al codice, viene mappata su file, ma in sola lettura, la parte alta, quella sotto heap, viene mappata con una modalità in scrittura particolare (MDR non ricordava il termine). Valori iniziali letti dal file senza propagare le letture/scritture su disco. La parte alta sotto l'heap è dedicata a variabili con contenuto nullo. Nell'eseguibile è solo segnata quanto è ampia quella zona, e il SO mappa quella zona sulla read-only static zero page.

Allocazione della memoria per il kernel

Il kernel implementa una serie di astrazioni. Ma il SO può sfruttarla per i suoi scopi? Sì, può usare gli spazi di indirizzamento virtuale per le proprie esigenze, ma a volte non è la soluzione ottimale. Da una parte, questa sorta di spreco (quale ?) non posso usare l'indirizzamento virtuale per alcuni scopi. Es. abbiamo parlato del DMA, alcuni di questi controller, non usando la CPU, non passano dalla MMU. Quindi bisogna fornire l'indirizzo fisico del buffer in cui le informazioni dovranno essere scritte. A prescindere dal fatto che gestire l'indirizzo fisico di un processo è più difficile, siamo limitati ad avere una sola pagina, o comunque dobbiamo gestire in modo contiguo la memoria fisica. In alcuni casi c'è la possibilità di fare meglio.

Slab allocator: viene usato perché sfrutta alcune peculiarità per una gestione della memoria quasi ottimale in termini di spreco. Idea: quando Linux ha bisogno di allocare/deallocare pagine, es. un record implementa un semaforo, oppure file aperti, altro esempio di oggetti che rappresentano entità diverse. PCB: 1k, e semaforo 1/4 di k. Come vengono gestite queste varie tipologie di oggetti? Si avrà una struttura di supporto, gestita con slab allocator, e si avrà una cache. Anche se qui cache significa altro. Cache: **insieme di slab**. Da non confondere con la cache per come la intendiamo noi. L'idea è quella che ho una struttura di supporto per il kernel formava da cache e da slab. Per ogni cache, ho una taglia per le strutture dati di cui ho bisogno. Es. supponiamo struttura dati da 1K, e ci saranno, per altre taglie da supportare, altrettante cache supportate ognuna per una data taglia. Ai fini dell'ottimizzazione, del funzionamento di questo meccanismo, è fondamentale. Fa sparire ogni fenomeno di frammentazione interna ed esterna. Ho bisogno di customizzare per ogni specifica taglia. Quindi devo specificare per ogni taglia? Sì, ma solo per le taglie effettivamente usate dal kernel. Un numero limitato. Tipologie chiuse. Questo tipo di lavoro non è pensabile per i processi utente: loro possono allocare un numero di byte multiplo, che può cambiare sempre, non come per il kernel. Strategia quindi customizzata sulla dimensione, senza avere troppi casi da gestire. Supponiamo di avere una determinata taglia da supportare. Slab: sequenza di frame contigui in memoria RAM. Uno slab deve essere: un multiplo della dimensione della pagina del frame (e per definizione è già vero) e deve essere anche un multiplo della dimensione della pagina che vogliamo supportare. Es. mi servono pagine da 3 KB, e ho pagine da 4 KB, allora posso prendere tre pagine, e quindi 12 KB, che è multiplo di 3. L'idea è che lo slab viene visto come una sequenza di pagine/frame. D'altro lato, posso vederlo come una sequenza di

frame di questo tipo. Slab gestito dinamicamente, blocchi allocati e disallocati. Se lo slab è tutto pieno? Ne procuro un altro? Individuo un'altra zona della memoria fisica, e individuo una sequenza di pagine, di frame, che rispecchia di nuovo i vincoli di molteplicità, e otteniamo la cache, un set di slab. Qualunque slot in qualunque slab va bene. Non ci sono problemi di adattabilità o di spreco, dato che ho richiesto proprio 3 KB, per esempio. Non ci sono problemi di frammentazione interna o esterna. Interna no perché ho richiesto 3 KB e sono quelli che mi servono, e ho un blocco da 3 KB. Esterna nemmeno perché avrò proprio lo spazio che mi servirà. Naturalmente, se ho bisogno di blocchi di dimensione diversa, dovrò andare in un'altra cache, con un altro set di slab di dimensione diversa. Se tutti gli slab si svuotano contemporaneamente... [Da rivedere].

File system

Il problema è gestire grandi quantità di informazioni, in modo persistente e condiviso tra più processi, Astrazione: file e directory.

File e directory visti come file speciali: alcune di queste implementazioni hanno dettagli in comune. Spesso ci sono però peculiarità con caratteristiche del file system che non riscontriamo su altri.

Sui sistemi UNIX abbiamo la famiglia ext dei file system, o BTRFS, o ZFS, file system molto moderno. Con caratteristiche che vedremo ma che sono molto all'avanguardia da questo punto di vista. Ogni file system può avere anche approcci e convenzioni diverse passando dall'uno all'altro. I primi avevano limiti anche sulla lunghezza. Es. i file system UNIX sono case sensitive.

Windows implementa un approccio case reiventive (tipo?), dove quindi usa maiuscole e minuscole, ma quando deve fare ricerche, non le considera, e questo a volte può portare a problemi.

Tipologie di file: video, immagini, testo etc. E per i file regolari esistono varie specializzazioni. Anche le directory stesse possono essere viste come file speciali. Directory contenitori di altri oggetti visti come file speciali. Abbiamo i file speciali di dispositivo che permettono di rappresentare, all'interno del file system, dei file speciali. Possiamo avere file di dispositivo speciale che riguarda il contenuto del mio disco, di una partizione, etc. Una scheda audio può essere vista come uno o più file principali.

Con la mappatura dei file in memoria posso fare cose molto potenti in modo semplice. Es: mappare un intero disco sullo spazio di indirizzamento. Distinguiamo documenti doc da estensioni. L'estensione è il meccanismo storico per riconoscere un file dal suo nome. Approccio classico di Windows. UNIX lo mantiene per compatibilità, ma preferisce un altro approccio: magic byte. Supponiamo di avere una sequenza di byte che rappresenta la mia immagine. I primi due byte del file sono magic byte, e vengono valorizzati con valori chiave che rappresentano la tipologia stessa. Windows si affida al nome mentre UNIX associa i magic byte per l'apposito tipo. Per MacOS, invece, al file è connesso sia il tipo sia l'applicazione da lanciare [da rivedere]. Magic byte legato anche ai file interpretabili. Ovvero file di testo che possono essere interpretati. **Shabang** visto nella parte di laboratorio. Prima riga del file di testo, con l'iniziale #!, e poi /bin/bash o, per esempio, /bin/python3.

In alcuni casi questo potrebbe mancare nel file ma essere comunque eseguibile. Se lo lanciamo da bash può creare problemi. In altre shell può funzionare. Però in uno script scritto bene va sempre scritto qual è l'interprete da utilizzare.

File system diversi presentano un set, un insieme di metadati diversi. Informazioni a contorno, satellite, dell'oggetto. Informazioni utili. SO diversi possono però avere convenzioni diverse.

Quando parliamo di lock su file, distinguiamo i lock di tipo shared e di tipo exclusive. Se andiamo a distinguere la tipologia di accesso (sola lettura o scrittura), possono essere rese compatibili fra di loro. Cioè, potenzialmente possono portare a problemi. Le stesse regole possono essere applicate ai file. Si parla di shared file quando si pensa a operazioni di lettura. Lock condiviso. Un SO che permette e distingue questi approcci, può fare sì che un processo acceda a un file insieme ad altri a patto che il lock e le intenzioni del processo siano di sola lettura.

Altro approccio, ortogonale a questa prima distinzione: modo in cui il lock viene applicato. Distinguiamo sistemi in cui ci sono mandatory lock piuttosto che lock di tipo advisory. Il SO può decidere se applicare gli effetti del lock. Cioè, può confermare di aver acquisito uno dei due tipi di lock, shared o exclusive, se li supporta entrambi. Che succede se arriva un processo P1 che ha lock di tipo esclusivo su un file. E se P2 arriva e cerca di aprire quel file? Se è di tipo mandatory, il SO lo blocca, e viene segnalato come un errore. È il SO stesso che si

occupa di fare da guardia, di bloccare eventuali operazioni. Può sembrare l'unico modo di applicare questa regola, ma ce n'è un altro: advisory. Permette di gestire il problema del lock, permette ai processi di acquisire e rilasciare i lock, ma fondamentale serve ai processi per capire se serve il lock per il processo. Se arriva P2 e cerca di accedere al file senza aver acquisito il lock, su un sistema di tipo advisory questa cosa è permessa. Meccanismo di tipo advisory da vedere come semaforo, verde e rosso. Mandatory invece è una sorta di sbarra che scende. Il lock in advisory diventa un sistema di coordinamento. Es. in un sistema mutex, se non acquisiamo il lock, dobbiamo bloccarci, ma non ci viene impedito. Una sorta di auto disciplina. Nel tipo advisory, usato per tipi di strutture che si sa essere condivise. Approcci diversi, legati a scelte storiche fatto sul tipo di SO.

Es. su windows c'è il mandatory: cercare di aprire un file che è già usato da un altro processo dà un errore visibile all'utente.

Su UNIX c'è l'advisory, il fatto di dover rispettare il lock spetta ai processi stessi.